Orbit Core — Scaling Plan

Rodrigo Menchio + Claude

2026-03-26

Orbit Core — Performance Scaling Plan

Current Architecture Summary

Component	Config
PostgreSQL 16	Single instance, 512MB container, 256MB shm
Connection Pool	Single pg.Pool, max 50 (shared API + workers)
Workers	In-process (setInterval), 5 background workers
Cache	In-memory only (5min TTL), no Redis
VPS	87–90% CPU steal time (oversold provider)
Partitioning	orbit_events + metric_points (monthly), rollup tables unpartitioned

Diagnosis: Bottlenecks

Bottleneck	Evidence
PG single-instance, 512MB RAM	shared_buffers limited, no read replica
Single pool (50 conn)	Workers (rollup, correlate, alerts, threat-intel) compete with API queries
Rollup tables not partitioned	metric_rollup_5m with 90 days = heavy scan on INSERT ON CONFLICT
Workers in-process	A heavy rollup blocks the Express event loop
VPS with 87–90% CPU steal	No software optimization can fix oversold hardware
Zero distributed cache	In-memory cache dies on restart, no sharing
Correlate worker	3 CTEs with STDDEV + JOIN every 5min over raw data

Phase 1 — Quick Wins (no architecture change)

1.1 Separate Pool: API vs Workers

Create two `pg.Pool` instances:

- **poolApi** (max 35): Serves all HTTP route handlers
- **poolWorker** (max 15): Serves rollup, correlate, alerts, threat-intel, connectors

Impact: Workers can never starve API queries. Pool saturation warnings become actionable per-pool.

1.2 Partition Rollup Tables

Convert `metric_rollup_5m` and `metric_rollup_1h` from plain tables to range-partitioned by `bucket_ts` (monthly).

Impact: INSERT ON CONFLICT scans only the current month's partition instead of the full 90/180-day table. Retention purge becomes O(1) DROP PARTITION.

1.3 Materialize Catalog Queries

Create a `catalog_event_cache` table refreshed by the rollup worker (every 5 min). The `/catalog/events` endpoint reads from cache instead of running 4 parallel GROUP BY queries over 30 days of events.

Impact: Eliminates the heaviest read query from the API hot path. Catalog response goes from ~500ms to ~5ms.

1.4 Increase PG Container Resources

- Container memory: 512MB → 1GB
- shm_size: 256MB → 512MB
- Enables larger shared_buffers, better cache hit ratio

Impact: More data stays in PG buffer cache, fewer disk reads.

Phase 2 — Separation of Concerns (medium term)

2.1 Workers as Separate Process

Extract rollup, correlate, and threat-intel workers into an independent `orbit-worker` Docker service. Own memory limit, own pool, no event loop contention with Express.

2.2 Redis Cache Layer

Add Redis for frequently-accessed data (catalog, wazuh summary, RAG catalog). Survives restarts, shareable across multiple API instances.

2.3 Read Replica

PG streaming replication: API reads from replica, workers write to primary. Doubles read capacity without touching query code.

2.4 PgBouncer

Connection pooler in front of PG (transaction mode). Hundreds of virtual connections with ~30 real PG connections. Essential for horizontal scaling.

Phase 3 — Real Scale (ambitious)

3.1 TimescaleDB

Drop-in PG replacement. Hypertables with native compression (10–20x disk savings), continuous aggregates replace manual rollups, automatic retention policies. Migration is nearly transparent.

3.2 Dedicated VPS

With 87–90% steal time, hardware is the real bottleneck. A dedicated 4 vCPU + 8GB RAM VPS (Hetzner/OVH ~€25/month) changes the game entirely.

3.3 Queue-Based Ingest (BullMQ + Redis)

Ingest endpoint accepts, enqueues, returns 202. Worker processes in optimized batches. Decouples API latency from write throughput.

3.4 Horizontal API Scaling

With Redis cache + PgBouncer + read replica, running 2–3 API instances behind Traefik is trivial. Stateless API, shared cache, pooled connections.

Priority Matrix

Item	Impact	Effort	Priority
1.1 Split pools	High	Low	**Now**
1.2 Partition rollups	High	Low	**Now**
1.3 Catalog cache	Medium	Low	**Now**
1.4 PG resources	Medium	Trivial	**Now**
2.1 Worker process	High	Medium	Next sprint
2.2 Redis	Medium	Medium	Next sprint
2.3 Read replica	High	Medium	Next sprint
2.4 PgBouncer	Medium	Low	Next sprint
3.1 TimescaleDB	Very High	High	Q2 2026
3.2 Dedicated VPS	Very High	Low	ASAP
3.3 Queue ingest	High	Medium	Q2 2026
3.4 Horizontal API	High	Low	After 2.x

Generated 2026-03-26 by Rodrigo Menchio + Claude Code